CSCI212: Lecture #16

Kevin Bedoya

Monday 3/30/26

Return to the NotesApp GUI

In this class, we will work on coding up the logic for *helper methods* in the **NotesApp** such as **handleWordCount()**, **handleCharCount()**, **handleAverage()**, and **handleClear()**.

handleAverage()

1. Collects text

```
    String text = area.getText();

```

2. Computes average

```
    double avg = averageWordLength(text);

```

3. reports it into a label

```
    status.setText(avg);

```

averageWordLength()

```
double averageWordLength(String text){
    text = text.trim();
    // String method that trims initial & final spaces; removes uncessary gaps/spaces at the beginning and ends of the input string
    String words[] = text.split("\\s+");
    // "\\s+" means "one or more spaces"
    // this is an example usage of a "regex" (this stands for "regular expression")
    // As a result, this the line above populates the words array with separate string objects
    int total = 0;
    for(String s : words){
        total += s.length();
    }
    int count = words.length;
    return total/(count * 1.0);
}
```

However, the above implementation does not properly handle the case when the text is empty! This will lead to a divide by zero exception (since count = 0 when the text is empty) and consequently crash the whole application! To fix this, we can include a conditional statement to throw an appropriate exception, but the GUI will still crash regardless. A better idea would be to *catch* the exception! That way, the user will still be able to operate on the GUI even after an exception was thrown. Let's include a *try-catch* block into **handleAverage()**

handleAverage() (more robust)

```
private void handleAverage(){
    try{
        String text = area.getText();
        double avg = avergeWordLength(text);
        status.setText(String.format("Average length: %.2f", avg));
    } catch (Exception e){
        //Gracefully handle the exception
        //This is catching the exception potentially being thrown in averageWordLength()
        status.setText("Error: " e.getMessage());
    }
}
```

averageWordLength() (more robust)

```
double averageWordLength(String text) throws Exception{
    text = text.trim();

    if(text.isEmpty()){
        throws new Exception("No words entered");
    }

    String words[] = text.split("\\s+");

    int total = 0;
    for(String s : words){
        total += s.length();
    }
    int count = words.length;
    return total/(count * 1.0);
}
```

For a more complex GUI, better organization would help. There is a standard technique called *Model-View separation*, a design technique used in GUI implementation. We separate the enterprise into two classes, a **model** class, and a **view** class. The *model* should include the mathematical/computational logic in response to actions, while the *view* should display the components that the user interacts with on the GUI. In **eclipse**, you are able to *refactor* your GUI code into *model* & *view* classes, a highly recommended practice to improve readability and organization of code (especially when the GUI is complicated!).

Updates to the NotesApp

The new notes app:

- Allows loading & saving file
- The new GUI will have 2 new buttons with responses.
- Will implement the **IOModel**

Refer to professor Ryba's website for the relevant coding exercises covered in this lecture.